

MedAssist AI

Medical Symptom Analysis & Disease Prediction System

MILESTONE 4 — COMPLETE PROJECT REPORT

Final End-to-End Implementation & Integration

Intern	Pochana Satish Kumar Reddy	Mentor	Jyoteesh (Data Scientist)
Internship	Infosys Springboard Internship	Project Phase	Final Project / Milestone 4
Degree	B.Tech — CSE (AI/ML)	Academic Year	2025–2026
Institution	Kalasalingam Academy of Research and Education	Project Type	AI-Powered Healthcare System

Final Project Summary

MedAssist AI is an end-to-end AI-powered healthcare decision-support web application that combines symptom analysis, machine-learning disease prediction, risk assessment, treatment recommendations, health reporting, medical history, analytics, and a dedicated administration portal.

Table of Contents

1. Objective
2. Project Overview
3. Problem Statement
4. Project Objectives
5. Work Completed in Previous Milestones
6. Final System Architecture
7. Technology Stack
8. Dataset & Machine Learning
9. Backend Development
10. Database Integration
11. Authentication & Security
12. Patient Application
13. Disease Prediction
14. Risk Assessment
15. Treatment Recommendation
16. Health Report Generation
17. Medical History & Profile
18. Patient Analytics
19. Administration Portal
20. Admin Analytics
21. API Integration & Testing
22. Frontend Integration
23. Git & GitHub
24. Challenges Faced
25. Final Module Status
26. Results
27. Limitations
28. Future Enhancements
29. Conclusion
30. References

1. Objective

The objective of Milestone 4 is to document the final and complete implementation of MedAssist AI. The project moves beyond the backend foundation documented in Milestone 2 and the frontend/integration phase to present the complete end-to-end healthcare application.

Milestone 4 consolidates the machine-learning prediction pipeline, REST APIs, PostgreSQL persistence, patient-facing React application, health reporting, analytics, and administrator portal into one final project deliverable.

2. Project Overview

MedAssist AI is a medical symptom analysis and disease prediction system designed as an AI-powered healthcare decision-support platform. The application allows a patient to enter symptoms and related health information, receive a model-based disease prediction with confidence information, assess risk, view treatment-oriented recommendations, maintain medical records, and generate health reports.

The final application also contains an administration portal for monitoring patients, reports, high-risk cases, system statistics, and analytics.

3. Problem Statement

Patients may need several disconnected tools to record health information, analyze symptoms, view predictions, maintain medical history, and obtain reports. Administrators similarly require a consolidated view of patient activity and health-system statistics.

MedAssist AI addresses this fragmentation by providing a single web-based platform with AI-assisted analysis, persistent records, reporting, analytics, and administration capabilities.

4. Project Objectives

The final system was developed around the following objectives:

- Integrate the trained disease-prediction model with a production-style REST API.
- Persist users, predictions, reports, and medical information using PostgreSQL and SQLAlchemy.
- Provide patient registration, login, profile management, symptom analysis, prediction, risk assessment, recommendations, records, reports, and analytics.
- Provide a separate administrator login and dashboard with patient, report, high-risk, and analytics views.
- Connect the React frontend to FastAPI APIs using Axios and complete the major end-to-end workflows.
- Test APIs using Swagger UI and debug frontend/backend integration issues.

5. Work Completed in Previous Milestones

Milestone 2 established the backend foundation. The reference milestone report records PostgreSQL database integration, authentication, Random Forest disease prediction, risk assessment, recommendations, PDF report generation, medical history, profile management, patient analytics, admin

operations, and Swagger testing as completed backend work. ■filecite■turn26file0■L13-L16■
■filecite■turn26file0■L17-L35■

The same milestone documented the completion of patient analytics, administrator functions, and admin analytics, including disease distribution, risk distribution, monthly prediction trends, common diseases, and common symptoms. ■filecite■turn26file0■L36-L44■

Milestone 4 therefore serves as the consolidated final report covering the backend foundation together with the completed frontend, integration, and final administration experience.

6. Final System Architecture

The final architecture is organized into a frontend presentation layer, FastAPI API layer, business and machine-learning layer, persistence layer, and reporting/static-file layer.

The React/Vite client communicates with FastAPI through REST endpoints. FastAPI validates requests, executes authentication and application logic, invokes the disease prediction model where required, persists records through SQLAlchemy, and returns structured JSON responses. Generated reports are stored and served through the backend.

PATIENT / ADMIN BROWSER
React + Vite + Bootstrap + Axios
REST API — FastAPI
Authentication Prediction Risk Recommendation Reports History Analytics Admin
Python ML Layer + Business Logic
scikit-learn pandas joblib Recommendation Data
PostgreSQL + SQLAlchemy

Figure 1. Final high-level architecture of MedAssist AI.

7. Technology Stack

The technologies used in the final project are:

- Frontend: React, Vite, React Router, Bootstrap, React Icons, Axios and charting components.
- Backend: Python, FastAPI, SQLAlchemy, Uvicorn and REST/OpenAPI.
- Database: PostgreSQL with SQLAlchemy ORM.
- Machine Learning: scikit-learn, pandas, NumPy and a trained Random Forest pipeline with joblib model artifacts.
- Security: JWT-based authentication and BCrypt password hashing as documented in the backend milestone.
- Reports: ReportLab for PDF health reports.

- Testing: FastAPI Swagger UI and browser-based frontend testing.
- Version Control: Git and GitHub.

8. Dataset & Machine Learning

The project dataset contains 4,920 training records, 42 testing records, 41 diseases, and 133 symptom features. The preprocessing phase examined missing values and duplicate rows and produced a cleaned training dataset for model integration.

The disease prediction workflow loads the trained model and label encoder. Selected symptoms are represented as a binary feature vector matching the model's symptom columns. The model returns a predicted disease, a probability-based confidence score, and the top three possible disease results.

The model output is presented as decision-support information and is not intended to replace professional medical diagnosis.

9. Backend Development

The FastAPI backend is divided into modular routers. The main application registers authentication, prediction, risk, recommendation, report, medical history, profile, analytics, and admin routes.

Static directories are configured for profile uploads and generated PDF reports. CORS is configured so the React development server can communicate with the FastAPI backend during development.

10. Database Integration

PostgreSQL provides persistent storage for application data, while SQLAlchemy ORM models provide a structured interface between Python application logic and relational tables.

The prediction workflow creates a prediction record containing a prediction identifier, patient identifier, selected symptoms, predicted disease, confidence, and later risk information. This enables subsequent report, history, and analytics workflows.

11. Authentication & Security

The system provides separate patient and administrator authentication flows. The backend uses token-based authentication for protected operations, while the frontend maintains session information required for authenticated navigation.

Password protection and role separation are important parts of the system design. Administrative functionality is separated from the patient application so that patient-facing workflows and administrative monitoring have distinct interfaces.

12. Patient Application

The patient application provides a shared navigation layout and a sequence of healthcare workflows. The major patient-facing pages include Dashboard, Health Analysis, Disease Prediction, Risk Assessment, Treatment Recommendation, Health Reports, Medical Records, Analytics, Settings, and Logout.

The interface uses React components and reusable styling to keep navigation and page structure consistent across the application.

13. Disease Prediction

The disease prediction workflow receives the patient identifier and selected symptoms. The backend transforms the symptom list into the model feature vector, executes the trained model, calculates confidence, stores the prediction, and returns the result.

The prediction is also stored for later use by risk assessment, reports, medical history, and analytics.

14. Risk Assessment

The risk assessment module evaluates prediction-related information and categorizes the result into Low, Medium, High, or Critical levels. The backend milestone specifically records these four risk categories as implemented. ■filecite■turn26file0■L23-L24■

The frontend presents the risk result as part of the patient's post-prediction healthcare workflow.

15. Treatment Recommendation

The recommendation module combines disease-related information with precautions, diet, lifestyle, medicine guidance, and related recommendation datasets. This extends the prediction workflow from classification toward practical decision-support information.

The recommendation content is presented as guidance and should not be interpreted as a substitute for a qualified clinician.

16. Health Report Generation

The report module generates professional PDF health reports containing patient information, prediction results, confidence, risk level, and recommendation information. Report generation using ReportLab was already documented as completed in the backend milestone. ■filecite■turn26file0■L28-L30■

The final frontend integrates report access into the patient workflow so that completed health information can be reviewed as a generated document.

17. Medical History & Profile

Completed predictions are retained as medical history so users can access previous health-related records. The backend milestone notes that completed predictions are stored with report references for future access. ■filecite■turn26file0■L31-L32■

Profile management supports patient information such as phone, address, emergency contact, and profile image, consistent with the previously documented implementation. ■filecite■turn26file0■L33-L35■

18. Patient Analytics

The patient analytics layer provides data-driven views of health activity, including disease distribution, risk distribution, and monthly prediction trends. These APIs were documented as completed in Milestone 2. ■filecite■turn26file0■L36-L38■

The final frontend uses these backend capabilities to present health information in a more understandable visual form.

19. Administration Portal

The final project includes a dedicated administrator interface with Admin Login, Admin Dashboard, Patients, High-Risk Patients, Reports, and Analytics pages.

The administrator workflow is designed for authorized users to monitor patient activity and system-level healthcare information without mixing administrative screens with the standard patient experience.

Administrative functions previously documented include viewing all patients, viewing an individual patient, searching patients, viewing reports, high-risk patient views, and dashboard statistics.

■filecite■turn26file0■L39-L41■



Figure 2. Representative MedAssist AI interface used as a visual reference for the final application documentation.

20. Admin Analytics

The administrator analytics module provides system-level statistics and visual analysis. The completed analytics scope includes disease distribution, risk distribution, monthly prediction trends, most common diseases, and most common symptoms. ■filecite■turn26file0■L42-L44■

The final admin interface organizes these statistics into dashboard cards and charts so that administrators can understand overall system activity.

21. API Integration & Testing

Swagger UI was used to test backend endpoints independently before and during frontend integration. The backend milestone identifies API testing as completed. ■filecite■turn26file0■L53-L66■

Frontend integration uses Axios requests from React components to FastAPI endpoints. During development, CORS and local development-port differences were identified and addressed as part of integration debugging.

22. Frontend Integration

The final React application connects the major patient and admin screens to backend services. Routing controls access to patient and administrator pages, while reusable layouts provide consistent navigation.

The integration phase also included handling loading states, API errors, authentication information, local session data, report access, and navigation between sequential healthcare workflows.

23. Git & GitHub

The project source code was maintained using Git and pushed to the GitHub repository branch used for development. The final workflow included staging frontend and backend changes, committing the implementation, pushing to the remote repository, and verifying repository status.

This provides version history and a recoverable source-code record for the completed project.

24. Challenges Faced

The project involved several technical challenges, particularly around connecting machine-learning inference with REST APIs, designing database relationships, maintaining authentication state, integrating reports with prediction history, and developing reusable analytics APIs. These challenges were explicitly identified in the backend milestone report. ■filecite■turn26file0■L67-L71■

During the final integration phase, additional practical debugging included frontend/backend CORS configuration, development-port changes, API error handling, navigation integration, and ensuring that the patient and administrator applications communicated with the correct backend.

25. Final Module Status

The following represents the final consolidated status of the project:

Module	Final Status
Database Integration	Completed
Patient Authentication	Completed
Admin Authentication	Completed
Disease Prediction	Completed
Risk Assessment	Completed
Treatment Recommendation	Completed
Health Reports	Completed
Medical History	Completed
Profile Management	Completed

Patient Analytics	Completed
Admin Dashboard	Completed
Admin Patients	Completed
High-Risk Patients	Completed
Admin Reports	Completed
Admin Analytics	Completed
Frontend Integration	Completed
API Testing	Completed
Git/GitHub Integration	Completed

26. Results

The final result is a complete AI-powered healthcare web application that brings together machine-learning prediction, patient workflows, persistent data storage, risk assessment, recommendations, reporting, analytics, and administrative monitoring.

The project demonstrates end-to-end development skills across AI/ML, Python backend development, relational database integration, REST APIs, React frontend development, authentication, visualization, document generation, testing, debugging, and version control.

27. Limitations

The system is a project-level decision-support application and must not be treated as a clinical diagnostic system. Model quality is dependent on the dataset and training process.

The current development environment is primarily local. A production deployment would require HTTPS, secure secret management, stronger authorization controls, monitoring, backups, and additional validation.

28. Future Enhancements

Potential future improvements include cloud deployment, production-grade security, clinician accounts, explainable AI, model monitoring, appointment scheduling, notifications, multilingual support, automated testing, stronger audit logs, and advanced longitudinal health analytics.

These enhancements can extend MedAssist AI from an academic prototype into a more scalable healthcare software platform.

29. Conclusion

Milestone 4 completes the project documentation for MedAssist AI. The system has evolved from a machine-learning prototype into an integrated healthcare application with a FastAPI backend, PostgreSQL persistence, React frontend, patient workflows, reporting, analytics, and an administrator portal.

The final implementation demonstrates how AI/ML can be combined with full-stack software engineering to create a structured healthcare decision-support platform. The project fulfills the major development objectives and provides a strong foundation for future production-oriented enhancements.

30. References

1. Milestone 2 Progress Report — MedAssist AI, dated 24 July 2026. ■filecite■turn26file0■L2-L7■
2. FastAPI and Swagger/OpenAPI documentation used during backend development and API testing.
3. React, Vite, React Router, Bootstrap and Axios documentation used for frontend development.
4. Python, pandas, scikit-learn, joblib and SQLAlchemy documentation used for data and backend implementation.
5. PostgreSQL and ReportLab documentation used for persistence and PDF report generation.